

READ FIRST

△**HEIGHT:** use the convention **PRINTED IN THE QUESTION**. 2026 papers say leaf = 0, empty subtree = −1 (your labs said leaf = 1).
ORDER: 12 → 10 → 7 → 5 → 4 → 11 → 1 → 2 → 6 → 3 → 9 → 8 → 13
Start with gradient descent: 15 pts of pure recall in five minutes.

T12 · GRADIENT DESCENT (15)

$\theta_{t+1} = \theta_t - \eta \cdot \nabla L(\theta_t)$
 θ_t = current parameters · $\eta > 0$ = learning rate · ∇L = gradient of the loss
Why minus: ∇L points to steepest **increase**; we want to decrease, so step the other way.
 η too small → very slow, many iterations. **η too large** → overshoots, oscillates or diverges.
Convex = graph lies at/below the chord between any two points = **no local min except the global one**. GD always finds the global min on a convex landscape, from any start. Non-convex → can stick in a local min.
 $L(w) = (w-a)^2 \rightarrow \nabla L(w) = 2(w-a)$
(w−3)², $\eta=0.5$, $w_0=0$: $w_1 = 0 - 0.5 \cdot 2(0-3) = \mathbf{3}$; $w_2 = \mathbf{3}$. Lands in ONE step.
(w−5)², $\eta=0.25$, $w_0=1$: $w_1 = 1+2 = \mathbf{3}$; $w_2 = 3+1 = \mathbf{4}$. Distance halves each step → asymptotic.
General: distance to a is multiplied by (1−2 η) per step.
batch = all n examples (stable, slow) · SGD = one (cheap, noisy) · mini-batch = compromise. Backprop = chain rule layer by layer.

T10 · HASH TABLES (10)

$h(k) = k \bmod p$, **separate chaining**. Draw **every** slot including empty ones. Chain order = insertion order.

Operation	Hash	AVL
Search	O(1) expected	O(log n)
Insert	O(1) amortised	O(log n)
Delete	O(1) expected	O(log n)
Sorted output	O(n log n)	O(n) in-order

Worst case $\Theta(n)$ — all keys in one chain (adversarial input, or all keys congruent mod p).
Hash wins on average speed; AVL wins on **worst case + sorted output**.

T7 · KRUSKAL / MST (20)

Repeatedly take the cheapest edge that creates no cycle. Cycle test = **find-union**: reject exactly when both endpoints are already in one component.
Table columns: **edge · Add/Reject · components after**.
△Must add exactly n−1 edges. Once all vertices are in one component, REJECT everything left.
 $\Theta(m \log m)$ — dominated by sorting the edges. Find-union is $\Theta(\alpha(n))$ amortised.
6 vertices, edges 3-4(1), 1-3(2), 2-3(3), 5-6(3), 3-5(4), 2-4(5), 1-2(6):
Add, Add, Add, Add, Add, Reject, Reject → weight 1+2+3+3+4 = **13**

T5 · DIJKSTRA + BFS (20)

△2026 graphs are **DIRECTED** — relax **OUTGOING** edges **only**.
Moves of size $s_1, s_2 \rightarrow dp[n] = dp[n-s_1] + dp[n-s_2]$
Why: the LAST move was either s_1 or s_2 — disjoint and exhaustive, so add.
Path: follow parents back from the target, reverse.
BFS part: counts **HOPS**, ignores weights. They disagree wherever a multi-hop cheap detour beats a single heavy edge — e.g. BFS d[2]=1 via the direct edge, Dijkstra d[2]=3 via 1→3→2.
Extraction order isn't insertion order: if d[3]=2 < d[2]=4, vertex 3 comes out first.

T4 · DP STAIR CLIMBING (15)

Moves of size $s_1, s_2 \rightarrow dp[n] = dp[n-s_1] + dp[n-s_2]$
Why: the LAST move was either s_1 or s_2 — disjoint and exhaustive, so add.
 $dp[0] = 1$ (one way to stand still — not 0). Fill small cases by hand where a big step is impossible.

n	0	1	2	3	4	5	6	7	8
1 or 2	1	1	2	3	5	8	13	21	34
1 or 3	1	1	1	1	2	3	4	6	9

1-or-2 is Fibonacci. **$\Theta(n)$** time, one left-to-right pass; $\Theta(1)$ space if you keep only the last few.
Steps of 1 or 4 → $dp[0..3] = 1, 1, 1, 1$ then $dp[n] = dp[n-1] + dp[n-4]$. Redo base cases every time.

T11 · PSEUDOCODE SKELETONS (15)

BFS
dist[s] = 0
parent[s] = NONE
q = new Queue; q.add(s)
while q not empty do
 v = q.extractOldest()
 for every edge (v,w) do
 if dist[w] not yet computed then
 dist[w] = dist[v] + 1
 parent[w] = v
 q.add(w)

DIJKSTRA
dist[s] = 0; dist[u] = ∞ for u ≠ s
q = new PriorityQueue; q.add(s,0)
while q not empty do
 v = q.extractMin()
 for every edge (v,w,c) do
 if dist[w] > dist[v] + c then
 dist[w] = dist[v] + c
 parent[w] = v
 q.add(w, dist[w])

Same shape: condition compares, body assigns, then push the neighbour. BFS adds 1 and tests "unseen"; Dijkstra adds c and tests "can I do better".
ORDERING scrambled steps: init → prepare structure → set source → **LOOP line** → extract → process neighbours → return. **△The loop/repeat line comes BEFORE the body lines it encloses.**
Kruskal order: empty MST → sort edges → init find-union → for each edge → if Find≠Find add+Union → else discard → return.

T1 · ASYMPTOTICS (10)

Constants → 1, convert, keep the fastest term.
1 < log n < (log n)^k < √n < n < n log n < n^2 < n^3 < 2^n < 3^n < n!

see	write
log(n^k)	k log n → $\Theta(\log n)$
(√2)^{log n}	2^{(log n)/2} = √n
√n log n + √n	√n(log n + 1) = $\Theta(\sqrt{n} \log n)$
n! vs 3^n	n! wins
2^n vs n^1000	2^n wins

Log base irrelevant inside Θ ; **exponential base is not**.
Counting loops
inner loop of CONSTANT length (y = x−10 to x+10) → **$\Theta(n)$**
for i=1..n / for j=1..n (triangular) → **$\Theta(n^2)$**
while y ≥ x counting down from 10n → **$\Theta(n^2)$**
while j ≤ n: j = j·2 (doubling) → **$\Theta(\log n)$** → nested = $\Theta(n \log n)$
innermost for k=1 to 10 → contributes nothing
1+2+...+n = n(n+1)/2 = $\Theta(n^2)$ · 1+2+4+...+2^n = $\Theta(2^{n+1})$
Tell for log: the variable is multiplied/divided, not incremented.

T2 · RECURRENCES (15)

Count calls EXACTLY:
while i > n-4 from i=n → i = n,n−1,n−2,n−3 → **4 calls**
while i ≤ n-3 from i=1 → **n−3 calls** (depends on n!)
for j=1 to 9 → 9 · final for k=1 to n^d → $\Theta(n^d)$
4+12 = 16 on [n/15] + $\Theta(n^6)$ → **$T(n) = 16T(\lfloor n/15 \rfloor) + \Theta(n^6)$**
(n−3)+9 = n+6 on n/6 + $\Theta(n^5)$ → $T(n) = (n+6)T(n/6) + \Theta(n^5)$
△branching factor depends on n → MASTER THEOREM DOES NOT APPLY. Say so.
MASTER: $T(n) = aT(n/b) + \Theta(n^d)$, $c = \log_b a$
 $d < c \rightarrow A \rightarrow \Theta(n^c)$ · $d = c \rightarrow B \rightarrow \Theta(n^c \log n)$ · $d > c \rightarrow C \rightarrow \Theta(n^d)$
Write a, b, f(n), c, case, result — all six.
2^k: 2, 4, 8, 16, 32, 64, 128, 256, 512, 1024 (k=1..10) · 3^k: 3, 9, 27, 81
8^T(n/2)+n^2→ $\Theta(n^3)A$ · 4T(n/2)+n^2→ $\Theta(n^2 \log n)B$ · 27T(n/3)+n^2→ $\Theta(n^3)A$ · 2T(n/2)+n→ $\Theta(n \log n)B$

T6 · HEAP + BST + AVL (15)

Heap: children of i = 2i, 2i+1; parent = [i/2]. Parent ≥ both children.
Answer = the index of the **CHILD**: new value = **any value ≤ the parent** △(and ≥ its own children, if it has any). For a 12-element array, indices 7–12 are leaves → no lower bound.
BST: left < v < right, all the way down. Never restructure. Repeated key → no change. **△CHECK: in-order traversal must come out sorted.**
AVL: at every node **|h(left) − h(right)| ≤ 1**. Write every height bottom-up. **△Use the question's convention (2026: leaf = 0, empty = −1).**

T3 · SORTING (15)

Bounded value range → COUNTING SORT. $\Theta(n+k)$. Say: **beats the $\Theta(n \log n)$ comparison bound because it never compares elements**.
MergeSort trace: show split levels down to singletons, then merge levels back up.
QuickSort trace: pivot ends in its final position, ≤ on the left, > on the right; recurse.
Lower-bound paragraph (learn it): a comparison sort is a decision tree — internal nodes are comparisons, leaves are permutations. All **n!** permutations must be reachable, so ≥ n! leaves, so height ≥ log₂(n!) = **$\Theta(n \log n)$** . Hence $\Omega(n \log n)$, and MergeSort attains it → optimal.
"O(n log log n)?" "O(nα(n))?" → **NO**, both are o(n log n). Always say **"comparison-based"**.

	worst	space	stable
insertion	n ² (n if sorted)	1	Y
merge	n log n	n	Y
quick	n² (avg n log n)	log n	N
heap	n log n	1	N
counting	n+k	n+k	Y
radix	d(n+k)	n+k	Y

T9 · DATA STRUCTURES (15) — 3 scenarios

scenario says	answer
highest priority, triage, scheduling	Max-heap / PQ
"same group?", merging groups	Find-union
worst-case log n + sorted output	AVL tree
O(1) average , no ordering ever	Hash table
strict FIFO, only enqueue/dequeue	Queue
static data, binary search, min memory	Sorted array
only ever add, never search	Unsorted array

Heap: **no key lookup**. Hash: **$\Theta(n)$ worst**, no sorted output. Unbalanced BST: degrades to $\Theta(n)$. Find-union: $\Theta(\alpha(n))$ amortised, for dynamic connectivity only.
WRITE 4 PARTS: name / how it fits / **time AND space** / why every other option loses here.
Seen: ER patients→heap · friend groups & bacteria→find-union · leaderboard & bookings→AVL · compiler symbol table→hash · HTTP requests→queue

T8 · P AND NP (15)

P = decision problems solvable deterministically in polynomial time.
NP = decision problems whose proposed solution (**certificate**) can be **verified** in polynomial time.
NP-complete = (1) in **NP** and (2) every NP problem reduces to it in poly time (NP-hard).
In P: sorting (merge) · shortest path $w \geq 0$ (Dijkstra) · MST (Kruskal/Prim) · binary search · **2-colouring/bipartite (BFS)** · connectivity
NP-complete: Hamiltonian cycle · **SAT (first, Cook-Levin 1971)** · knapsack (decision) · independent set · vertex cover · **3-colouring** · TSP **△2-colouring is P but 3-colouring is NP-c**. **Shortest path is P but Hamiltonian path is NP-c**.
TRUE: X NP-c ∧ X ∈ P → P=NP · P ⊆ NP · every NP problem reduces to any NP-c · every NP-c reduces to every other NP-c · SAT is NP-c (and was the first)
FALSE: parallel processors solve NP-c in poly time · NP-c can only be approximated · all NP needs exponential time · Hamiltonian cycle is in P
A/B reduction pattern (A in P, B NP-c): $A \rightarrow B$ reduces to $B \rightarrow A$ in P · unknown if B reduces to A · $A \rightarrow$ unknown if B in P. Anything in P reduces to anything.

T13 · ALGORITHM DESIGN (10)

Single pass. **$\Theta(n)$ time, $\Theta(1)$ space** — always justify both in one line.
SLIDING WINDOW (fixed k):
windowSum = Σ w[1..k]
for i = 2 to n-k+1:
 windowSum = windowSum − w[i-1] + w[i+k-1]
 if windowSum < bestSum:
 bestSum = windowSum; bestStart = i
△new = old − (element leaving) + (element entering)
LONGEST RUN:
bestLen = currLen = 1; bestStart = currStart = 1
for i = 2 to n:
 if T[i] > T[i-1]:
 currLen = currLen + 1
 if currLen > bestLen:
 bestStart = currStart; bestLen = currLen
 else:
 currStart = i; currLen = 1
Variants only change the comparison. Init $\Theta(k)$ then $O(1)$ per step.

POSSIBLE EXTRAS (practice exam)

Extra memory (not counting input): merge sort $\Theta(n)$ · binary search $\Theta(1)$ · heap-sort $\Theta(1)$ · Dijkstra $\Theta(n+m)$ → $\Theta(n)$ sparse, **$\Theta(n^2)$** when m = $\Theta(n^2)$
Composite times: build-heap + n removeMax = $\Theta(n \log n)$ · Floyd-W. inside a double loop = $\Theta(n^3)$ · n inserts into a sorted array = **$\Theta(n^2)$** (shifting, not searching) · 100n² AVL adds = $\Theta(n^2 \log n)$ since log(100n²) = $\Theta(\log n)$
Sort graph edges in $O(n+m)$: radix = two counting sorts, by target then by source; stability preserves the first pass. (BFS is the wrong answer).
Python vs C++ ordering: asymptotics first, language second. Removing from the **front** is $\Theta(n)$ each → $\Theta(n^2)$; from the **back** is $O(1)$ amortised → $\Theta(n)$.
Memoised → DP: dictionary → array, add a loop over the index from small to large, recursive calls → array lookups, drop the guard, return the last entry.

FINAL CHECKS

1. Heights — did you use the convention printed in the question?
2. Kruskal — exactly n−1 edges added?
3. Dijkstra — dist[w] = dist[parent[w]] + edge cost, for every w?
4. BST — does the in-order traversal come out sorted?
5. Recurrence — is the branching factor constant? If not, say the Master theorem fails.
6. Task 9 — did you write why the OTHER structures lose?
7. dp[0] = 1, not 0.
8. Master theorem — a, b, f(n), c, case, result. All six.